

SHIP CLASSIFICATION

MANAGEMENT SYSTEM

Tutoriel Complet DevOps sur Azure

Kubernetes · AKS · OpenShift · JFrog · ArgoCD · cert-manager · Ingress · APIM



Contexte Métier



Architecture Cible



Tutoriel Pas-à-Pas



Backlog Produit



Planning Sprints



Estimation Efforts

Guide complet pour déployer un système de gestion des navires et leurs dossiers de classification en utilisant la stack DevOps Azure : de l'infrastructure au déploiement applicatif, avec backlog détaillé, planning sur 6 sprints et estimations en points d'effort.

PARTIE 1 — Contexte Métier & Architecture Cible

Présentation du domaine métier : gestion des navires, dossiers de classification, acteurs et flux de données. Définition de l'architecture technique cible sur Azure.

1.1 — Présentation du Domaine Métier

Le système ShipClass gère le cycle de vie complet des navires et de leurs dossiers de classification maritime. Les sociétés de classification (Bureau Veritas, Lloyd's Register, DNV...) émettent des certificats qui attestent de la conformité des navires aux normes internationales (SOLAS, MARPOL, ISM Code). Ce système centralise les informations navires, les inspections, les certificats, les rapports de surveyors et les alertes de non-conformité.

Acteur	Rôle	Actions dans le système
Armateur	Propriétaire / opérateur du navire	Consulter les dossiers, soumettre des demandes d'inspection, télécharger les certificats
Surveyor (Inspecteur)	Expert de classification	Créer des rapports d'inspection, émettre des recommandations, valider les travaux
Officier de classification	Responsable du dossier	Approuver les dossiers, émettre les certificats, gérer les non-conformités
Administrateur IT	Gestion de la plateforme	Déployer les mises à jour, gérer les secrets, surveiller les métriques
API externe (AIS)	Système d'identification automatique	Pousser la position GPS et les données techniques des navires en temps réel

1.2 — Entités Métier Principales

Entité	Attributs clés	Relations
Navire (Vessel)	IMO Number, nom, pavillon, type, GRT, date de construction, propriétaire	1 navire → N dossiers de classification
Dossier de Classification	Numéro de dossier, classe, statut, date d'échéance, société de classification	1 dossier → N certificats + N inspections
Certificat	Type (Safety, Load Line, IOPP...), numéro, date émission, date expiration, autorité	Lié à 1 dossier, déclenche alertes d'expiration
Inspection	Date, lieu (port), type (annuel/quinquennal), surveyor, constatations	Liée à 1 dossier, génère 0..N non-conformités
Non-Conformité	Code NC, description, gravité (obs/minor/major/ISM), délai de correction	Liée à 1 inspection, statut ouvert/clôturé
Document	Type, fichier (PDF/CAD), version, auteur, date dépôt	Lié à 1 dossier ou 1 inspection

1.3 — Architecture Technique Cible

Le système est décomposé en microservices containerisés déployés sur AKS (production) et ARO/OpenShift (environnement réglementé). JFrog Artifactory centralise les images Docker et charts Helm. ArgoCD assure la synchronisation GitOps. Azure APIM expose les APIs aux partenaires externes. cert-manager gère les certificats TLS automatiquement.

Microservice	Technologie	Port	Base de données	Rôle
vessel-service	Node.js / Express	3001	PostgreSQL (vessels DB)	CRUD navires, intégration AIS

Microservice	Technologie	Port	Base de données	Rôle
classification-service	Java / Spring Boot	3002	PostgreSQL (class DB)	Gestion dossiers, workflow d'approbation
certificate-service	Python / FastAPI	3003	PostgreSQL (cert DB)	Émission certificats, alertes expiration
inspection-service	Node.js / Express	3004	PostgreSQL (inspect DB)	Rapports inspection, non-conformités
document-service	Python / FastAPI	3005	Azure Blob + PostgreSQL	Upload/Download docs, versioning
notification-service	Node.js	3006	Redis (cache) + Queue	Alertes email/Teams, Service Bus
api-gateway	Azure APIM	443	—	Auth OAuth2, rate-limit, routage
frontend	React / TypeScript	80	—	Interface web responsive


```
##### # Project Settings → Service Connections → Docker Registry # URL :
https://artifactory.shipclass.example.com # Username / Password : robot-shipclass / <token>
```

■ Résultat attendu

JFrog Artifactory opérationnel avec dépôts docker-shipclass-local et helm-shipclass-local. Xray activé pour le scan CVE automatique à chaque push d'image.

4 Déployer la Base de Données PostgreSQL avec Private Endpoint

```
# ■■ 4.1 Créer PostgreSQL Flexible Server ##### az postgres flexible-server create `
--resource-group $RG ` --name postgres-shipclass ` --location $LOCATION ` --admin-user pgadmin ` --admin-password $(az
keyvault secret show --vault-name $KV_NAME ` --name pg-admin-password --query value -o tsv) ` --sku-name
Standard_D4s_v3 ` --version 15 ` --public-access None ` --vnet $VNET_NAME ` --subnet db-subnet # ■■ 4.2 Créer les
bases de données ##### foreach ($db in
@("vessels_db","classification_db","certificates_db","inspections_db","documents_db")) { az postgres flexible-server
db create ` --resource-group $RG ` --server-name postgres-shipclass ` --database-name $db } # ■■ 4.3 Stocker la
connection string dans Key Vault ##### $PG_CONN =
"postgresql://pgadmin:<pw>@postgres-shipclass.postgres.database.azure.com:5432" az keyvault secret set --vault-name
$KV_NAME ` --name pg-vessel-conn --value "$PG_CONN/vessels_db?sslmode=require" az keyvault secret set --vault-name
$KV_NAME ` --name pg-class-conn --value "$PG_CONN/classification_db?sslmode=require" az keyvault secret set
--vault-name $KV_NAME ` --name pg-cert-conn --value "$PG_CONN/certificates_db?sslmode=require" az keyvault secret set
--vault-name $KV_NAME ` --name pg-inspect-conn --value "$PG_CONN/inspections_db?sslmode=require" # ■■ 4.4 Service
ExternalName K8s (alias DNS interne) ##### kubectl apply -f - <<EOF apiVersion: v1 kind: Service metadata: name:
postgres-svc namespace: shipclass-prod spec: type: ExternalName externalName:
postgres-shipclass.postgres.database.azure.com EOF
```

■ Résultat attendu

PostgreSQL Flexible Server avec 5 bases de données, accessible uniquement via le VNet privé. Toutes les connection strings stockées dans Azure Key Vault. Les pods se connectent via 'postgres-svc:5432'.

5

Configurer les Secrets avec Azure Key Vault + External Secrets Operator

```
# ■■ 5.1 Créer la Managed Identity pour l'application ■■■■■■■■■■ az identity create --resource-group $RG --name
mi-shipclass-app $CLIENT_ID = az identity show -g $RG -n mi-shipclass-app --query clientId -o tsv $PRINCIPAL_ID = az
identity show -g $RG -n mi-shipclass-app --query principalId -o tsv $OIDC_URL = az aks show -g $RG -n $AKS_NAME `
--query oidcIssuerProfile.issuerUrl -o tsv # ■■ 5.2 Donner accès au Key Vault ■■■■■■■■■■ az
role assignment create ` --role "Key Vault Secrets User" ` --assignee $PRINCIPAL_ID ` --scope $(az keyvault show -g
$RG -n $KV_NAME --query id -o tsv) # ■■ 5.3 Federated Credential (lien AAD ↔ K8s ServiceAccount) ■ az identity
federated-credential create ` --name fc-shipclass ` --identity-name mi-shipclass-app ` --resource-group $RG ` --issuer
$OIDC_URL ` --subject "system:serviceaccount:shipclass-prod:shipclass-sa" # ■■ 5.4 Installer External Secrets
Operator ■■■■■■■■■■ helm repo add external-secrets https://charts.external-secrets.io helm install
external-secrets external-secrets/external-secrets ` --namespace external-secrets --create-namespace # ■■ 5.5
ClusterSecretStore ■■■■■■■■■■ kubectl apply -f - <<EOF apiVersion:
external-secrets.io/v1beta1 kind: ClusterSecretStore metadata: name: azure-keyvault spec: provider: azurekv: authType:
WorkloadIdentity vaultUrl: https://kv-shipclass.vault.azure.net serviceAccountRef: name: external-secrets-sa
namespace: external-secrets EOF # ■■ 5.6 ExternalSecret pour vessel-service ■■■■■■■■■■ kubectl apply
-f - <<EOF apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: vessel-secrets namespace:
shipclass-prod spec: refreshInterval: 1h secretStoreRef: name: azure-keyvault kind: ClusterSecretStore target: name:
vessel-secrets data: - secretKey: DATABASE_URL remoteRef: key: pg-vessel-conn - secretKey: AIS_API_KEY remoteRef: key:
ais-api-key - secretKey: JWT_SECRET remoteRef: key: jwt-secret EOF
```

■ Résultat attendu

External Secrets Operator synchronise les secrets depuis Key Vault toutes les heures. Les pods accèdent aux secrets via des variables d'environnement classiques sans jamais voir les valeurs brutes dans Git.

6

Construire et Pousser les Images Docker vers JFrog

Exemple avec le microservice vessel-service (Node.js). Même structure pour tous les autres services.

```
# ■■ 6.1 Dockerfile – vessel-service ■■■■■■■■■■ # Dockerfile FROM node:20-alpine AS builder
WORKDIR /app COPY package*.json ./ RUN npm ci --only=production COPY src/ ./src/ FROM node:20-alpine RUN addgroup -S
appgroup && adduser -S appuser -G appgroup WORKDIR /app COPY --from=builder /app . USER appuser EXPOSE 3001
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:3001/health || exit 1 CMD ["node",
"src/index.js"] # ■■ 6.2 Build & Push via Azure DevOps pipeline ■■■■■■■■■■ # azure-pipelines.yml trigger:
branches: include: [main] paths: include: [services/vessel-service/*] variables: SERVICE_NAME: vessel-service
JFROG_REPO: docker-shipclass-local stages: - stage: Build jobs: - job: BuildAndPush pool: vmImage: ubuntu-latest
steps: - task: Docker@2 displayName: Build & Push to JFrog inputs: containerRegistry: JFrog-ShipClass repository:
$(JFROG_REPO)/$(SERVICE_NAME) command: buildAndPush dockerfile: services/vessel-service/Dockerfile tags: |
$(Build.BuildId) latest - script: | curl -fL https://install-cli.jfrog.io | sh jfrog config add jfrog-server
--url=$(JFROG_URL) --access-token=$(JFROG_TOKEN) jfrog xr build-scan $(Build.DefinitionName) $(Build.BuildId) --fail=true
displayName: Xray Security Scan env: JFROG_TOKEN: $(JFROG_TOKEN)
```

7

Déployer les Microservices sur AKS (Manifests K8s)

Exemple complet pour vessel-service. Répliquer la même structure pour chaque microservice.

```
# k8s/vessel-service/deployment.yaml apiVersion: apps/v1 kind: Deployment metadata: name: vessel-service namespace:
shipclass-prod labels: app: vessel-service version: v1 spec: replicas: 3 selector: matchLabels: app: vessel-service
strategy: type: RollingUpdate rollingUpdate: maxUnavailable: 1 maxSurge: 1 template: metadata: labels: app:
vessel-service azure.workload.identity/use: "true" spec: serviceAccountName: shipclass-sa imagePullSecrets: - name:
jfrog-registry containers: - name: vessel-service image:
artifactory.shipclass.example.com/docker-shipclass-local/vessel-service:latest ports: - containerPort: 3001 envFrom:
- configMapRef: name: shipclass-config - secretRef: name: vessel-secrets resources: requests: cpu: 250m memory: 256Mi
limits: cpu: 500m memory: 512Mi livenessProbe: httpGet: path: /health port: 3001 initialDelaySeconds: 30
periodSeconds: 15 readinessProbe: httpGet: path: /ready port: 3001 initialDelaySeconds: 10 periodSeconds: 5
securityContext: runAsNonRoot: true runAsUser: 1000 readOnlyRootFilesystem: true allowPrivilegeEscalation: false --- #
k8s/vessel-service/service.yaml apiVersion: v1 kind: Service metadata: name: vessel-svc namespace: shipclass-prod
spec: selector: app: vessel-service ports: - port: 80 targetPort: 3001 type: ClusterIP --- #
k8s/vessel-service/hpa.yaml apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: vessel-hpa
namespace: shipclass-prod spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: vessel-service minReplicas:
3 maxReplicas: 10 metrics: - type: Resource resource: name: cpu target: type: Utilization averageUtilization: 70
```

8

Configurer cert-manager & Ingress NGINX avec TLS Automatique

9 Configurer Azure APIM pour l'Exposition des APIs Partenaires

10 Mettre en Place GitOps avec ArgoCD

■ Résultat attendu

ArgoCD surveille en continu le repo k8s-config. Tout commit sur main déclenche un déploiement automatique. Le self-heal protège contre les modifications manuelles non autorisées. UI accessible sur <https://argocd.shipclass.example.com>

Accès Local à la Base de Données (Développement & Debug)

Monitoring, Alertes & Vérification Finale

L'ensemble du système ShipClass est déployé : 6 microservices avec HPA, base de données privée, secrets synchronisés depuis Key Vault, HTTPS automatique via cert-manager, APIM pour les partenaires, et GitOps ArgoCD pour les déploiements continus. Durée totale d'installation : ~2h.

PARTIE 3 — Backlog Produit (Product Backlog)

Liste complète des User Stories classées par Epic, avec priorité MoSCoW, estimation en points (Fibonacci), et critères d'acceptation. 47 User Stories réparties sur 6 Epics.

EPIC-1 — Gestion des Navires

EPIC-2 — Dossiers de Classification

EPIC-3 — Certificats & Alertes

EPIC-4 — Inspections & Non-Conformités

EPIC-5 — Documents & Fichiers

EPIC-6 — Infrastructure & DevOps

EPIC-1 — Gestion des Navires

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-001	En tant qu'armateur, je veux créer une fiche navire avec IMO, nom, pavillon, type et caractéristiques techniques	Must	8	Fiche navire créée avec validation IMO unique, retour 201 avec l'ID généré
US-002	En tant qu'officier, je veux rechercher un navire par IMO, nom ou pavillon avec pagination	Must	5	Résultats filtrés et paginés retournés en < 500ms
US-003	En tant qu'armateur, je veux mettre à jour les informations d'un navire (sauf IMO)	Must	3	Modification enregistrée avec audit trail (qui/quand)
US-004	En tant qu'officier, je veux consulter l'historique complet d'un navire	Should	5	Historique chronologique de tous les événements liés au navire
US-005	En tant que système AIS, je veux pousser la position et l'état du navire via webhook	Should	8	Endpoint POST /vessels/{imo}/position, validation token, mise à jour temps réel
US-006	En tant qu'armateur, je veux exporter la liste de mes navires en CSV/PDF	Could	3	Export généré et téléchargeable en < 30s pour jusqu'à 1000 navires

EPIC-2 — Dossiers de Classification

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-007	En tant qu'officier, je veux créer un dossier de classification lié à un navire avec une classe et une société	Must	8	Dossier créé avec numéro auto-généré, statut DRAFT, lié au navire
US-008	En tant qu'officier, je veux faire progresser le dossier dans un workflow (Draft→Review→Approved→Active)	Must	13	Transitions de statut validées, notifications envoyées à chaque changement
US-009	En tant que surveyor, je veux consulter les dossiers qui me sont assignés	Must	5	Liste filtrée par surveyor assigné, triée par priorité/date d'échéance
US-010	En tant qu'officier, je veux assigner un surveyor à un dossier	Must	3	Assignation enregistrée, notification email au surveyor
US-011	En tant qu'armateur, je veux suivre le statut de mes dossiers en temps réel	Should	5	Dashboard avec statuts actualisés, alertes sur changements
US-012	En tant qu'officier, je veux clôturer un dossier avec un motif (expiration, retraite, transfert)	Should	5	Clôture avec archivage, dossier non modifiable post-clôture
US-013	En tant qu'armateur, je veux transférer un dossier vers une autre société de classification	Could	8	Processus de transfert avec validation des deux parties

EPIC-3 — Certificats & Alertes

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-014	En tant qu'officier, je veux émettre un certificat lié à un dossier avec type, numéro et dates	Must	8	Certificat créé et stocké, PDF généré automatiquement
US-015	En tant que système, je veux envoyer une alerte 90/60/30/7 jours avant expiration d'un certificat	Must	13	Job planifié, emails envoyés aux armateurs et officiers concernés
US-016	En tant qu'officier, je veux renouveler un certificat expirant (crée une nouvelle version)	Must	5	Nouveau certificat lié à l'ancien, historique préservé
US-017	En tant qu'armateur, je veux télécharger mes certificats valides en PDF	Must	3	Téléchargement sécurisé avec watermark 'VALID' ou 'EXPIRED'
US-018	En tant qu'officier, je veux suspendre un certificat avec motif (incident, non-conformité majeure)	Should	5	Suspension enregistrée, notification immédiate à l'armateur
US-019	En tant que surveyor, je veux voir un tableau de bord des certificats expirant dans les 30 prochains jours	Should	5	Dashboard filtrable par société, type de certificat, navire
US-020	En tant qu'officier, je veux générer un rapport consolidé de tous les certificats actifs par navire	Could	5	Rapport PDF multi-pages avec sommaire et QR codes de vérification

EPIC-4 — Inspections & Non-Conformités

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-021	En tant que surveyor, je veux créer un rapport d'inspection avec date, lieu, type et constatations	Must	8	Rapport créé avec pièces jointes (photos), numéro auto-généré
US-022	En tant que surveyor, je veux enregistrer des non-conformités avec code, gravité et délai de correction	Must	8	NC créée avec statut OPEN, notification à l'armateur
US-023	En tant qu'armateur, je veux soumettre la preuve de correction d'une non-conformité	Must	5	Documents uploadés, NC passe en PENDING_REVIEW
US-024	En tant que surveyor, je veux valider ou rejeter la correction d'une non-conformité	Must	5	NC passe en CLOSED ou retourne en OPEN avec commentaire
US-025	En tant qu'officier, je veux voir toutes les NC ouvertes dépassant leur délai de correction	Must	5	Tableau de bord avec escalade automatique après dépassement
US-026	En tant que surveyor, je veux planifier une inspection future sur un dossier	Should	3	Inspection planifiée avec rappel 7 jours avant la date
US-027	En tant qu'officier, je veux générer un rapport synthétique des NC par navire/période	Should	8	Rapport statistique avec tendances, graphiques, taux de clôture
US-028	En tant que surveyor, je veux documenter une inspection à bord via application mobile (offline-first)	Could	21	Mode offline avec synchronisation différée, photos, formulaires

EPIC-5 — Documents & Fichiers

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-029	En tant qu'utilisateur, je veux téléverser des documents (PDF, DWG, images) liés à un dossier ou inspection	Must	8	Upload sécurisé vers Azure Blob, métadonnées indexées, < 50MB
US-030	En tant qu'utilisateur, je veux consulter et télécharger les documents d'un dossier	Must	5	Accès basé sur rôle, URL signée temporaire (24h)
US-031	En tant que surveyor, je veux créer une nouvelle version d'un document existant	Should	5	Versioning (v1, v2...), ancienne version archivée et accessible
US-032	En tant qu'officier, je veux rechercher dans le contenu des documents PDF (full-text search)	Should	13	OCR + indexation Azure Cognitive Search, résultats en < 2s
US-033	En tant qu'administrateur, je veux définir une politique de rétention par type de document	Could	5	Archivage automatique après N années selon règles métier

EPIC-6 — Infrastructure & DevOps

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-034	En tant que DevOps, je veux un pipeline CI/CD qui build, scanne et déploie chaque microservice	Must	13	Pipeline Azure DevOps complet : build → scan Xray → push JFrog → ArgoCD sync

ID	User Story	MoSCoW	Points	Critères d'acceptation
US-035	En tant que DevOps, je veux que les secrets soient gérés via Azure Key Vault sans secrets dans Git	Must	8	External Secrets Operator configuré, aucun secret en clair dans les repos
US-036	En tant que DevOps, je veux des certificats TLS automatiques sur tous les endpoints HTTPS	Must	5	cert-manager + Let's Encrypt, renouvellement automatique 15j avant expiration
US-037	En tant que DevOps, je veux que les APIs soient exposées via Azure APIM avec auth JWT	Must	8	APIM configuré, validate-jwt, rate-limiting, monitoring Application Insights
US-038	En tant que DevOps, je veux un monitoring avec alertes sur les métriques critiques	Must	8	Container Insights, alertes CPU/mémoire/erreurs 5xx, tableau de bord
US-039	En tant que DevOps, je veux une stratégie de sauvegarde de la base de données (RPO 1h, RTO 4h)	Must	5	Backup automatique PostgreSQL toutes les heures, restauration testée
US-040	En tant que DevOps, je veux des NetworkPolicies pour isoler les microservices	Should	5	Chaque microservice n'accepte que le trafic des sources autorisées
US-041	En tant que DevOps, je veux pouvoir accéder à la BDD depuis mon poste local pour le debug	Should	3	Script PowerShell port-forward documenté et testé
US-042	En tant que DevOps, je veux une stratégie de disaster recovery avec ArgoCD	Should	5	Procédure de restauration complète en < 4h depuis un repo Git
US-043	En tant que DevOps, je veux des tests de charge automatisés avant chaque mise en production	Could	8	k6 dans le pipeline CI, seuil : 500 req/s avec p99 < 200ms

PARTIE 4 — Planning des Sprints (6 Sprints × 2 Semaines)

Planning détaillé sur 12 semaines (3 mois) : 6 sprints de 2 semaines. Chaque sprint a un objectif clair, une liste de User Stories et une capacité en points d'effort. Basé sur une équipe de 5 développeurs + 1 DevOps + 1 PO.

SPRINT
1

Infrastructure & Fondations

Semaines 1-2

Objectif : Mettre en place toute l'infrastructure Azure (AKS, JFrog, PostgreSQL, Key Vault, ArgoCD, cert-manager) et le pipeline CI/CD de base. À la fin de ce sprint, un microservice vide déployable de bout en bout.

ID	User Story	Pts	Équipe
US-034	Pipeline CI/CD Azure DevOps → JFrog → ArgoCD	13	DevOps
US-035	Secrets via Azure Key Vault + External Secrets Operator	8	DevOps
US-036	Certificats TLS automatiques cert-manager	5	DevOps
US-039	Sauvegarde BDD (RPO 1h)	5	DevOps
US-040	NetworkPolicies isolation microservices	5	DevOps
US-037	Azure APIM avec auth JWT	8	DevOps
INFRA	Setup AKS, JFrog, PostgreSQL, VNet	11	DevOps
TOTAL		55	Capacité: 55 pts

SPRINT
2

Gestion des Navires (Core)

Semaines 3-4

Objectif : Implémenter les fonctionnalités de base de gestion des navires : CRUD complet, recherche et intégration AIS. Les armateurs peuvent créer et consulter leurs navires via l'API.

ID	User Story	Pts	Équipe
US-001	Créer une fiche navire (IMO, nom, pavillon...)	8	Dev 1+2
US-002	Rechercher un navire (pagination, filtres)	5	Dev 1
US-003	Mettre à jour les informations navire	3	Dev 2
US-004	Historique complet d'un navire	5	Dev 1
US-005	Webhook AIS pour position/état	8	Dev 3
US-029	Upload documents (PDF, images)	8	Dev 4
US-030	Consulter/télécharger documents	5	Dev 4
US-041	Script port-forward BDD local	3	DevOps
US-038	Monitoring alertes métriques	8	DevOps
TOTAL		53	Capacité: 52 pts

SPRINT

3

Dossiers de Classification

Semaines 5-6

Objectif : Implémenter le workflow complet de gestion des dossiers de classification : création, transitions de statut, assignation des surveyors. Les officiers peuvent piloter tout le cycle de vie d'un dossier.

ID	User Story	Pts	Équipe
US-007	Créer un dossier de classification	8	Dev 1+2
US-008	Workflow dossier (Draft→Review→Approved→Active)	13	Dev 1+3
US-009	Consulter les dossiers assignés (surveyor)	5	Dev 2
US-010	Assigner un surveyor à un dossier	3	Dev 2
US-011	Suivi statut dossiers en temps réel	5	Dev 3
US-031	Versioning documents	5	Dev 4
US-042	Disaster recovery ArgoCD	5	DevOps
US-006	Export liste navires CSV/PDF	3	Dev 5
US-012	Clôturer un dossier	5	Dev 5
TOTAL		52	Capacité: 52 pts

SPRINT

4

Certificats & Inspections

Semaines 7-8

Objectif : Implémenter l'émission des certificats avec alertes d'expiration automatiques, et le workflow complet des inspections et non-conformités. Le système peut alerter proactivement avant les expirations.

ID	User Story	Pts	Équipe
US-014	Émettre un certificat (type, dates, PDF)	8	Dev 1+2
US-015	Alertes 90/60/30/7j avant expiration	13	Dev 3
US-016	Renouveler un certificat	5	Dev 2
US-017	Télécharger certificats en PDF	3	Dev 2
US-021	Créer rapport d'inspection	8	Dev 4+5
US-022	Enregistrer non-conformités (code, gravité)	8	Dev 4
US-023	Soumettre preuve de correction NC	5	Dev 5
US-024	Valider/rejeter correction NC	5	Dev 5
TOTAL		55	Capacité: 55 pts

SPRINT

5

Fonctionnalités Avancées & Reporting

Semaines 9-10

Objectif : Ajouter les fonctionnalités Should/Could : tableau de bords, rapports, recherche full-text documents, gestion des NC en retard. Le système offre une vision analytique complète.

ID	User Story	Pts	Équipe
US-018	Suspendre un certificat avec motif	5	Dev 1
US-019	Dashboard certificats expirant 30j	5	Dev 1
US-025	Dashboard NC ouvertes dépassant délai	5	Dev 2
US-026	Planifier une inspection future	3	Dev 3
US-027	Rapport synthétique NC par navire/période	8	Dev 3+5
US-032	Recherche full-text dans documents PDF	13	Dev 4
US-020	Rapport consolidé certificats par navire	5	Dev 5
US-013	Transfert dossier entre sociétés	8	Dev 2
TOTAL		52	Capacité: 48 pts

SPRINT
6

Finalisation, Tests & Mise en Production

Semaines 11-12

Objectif : Tests de charge, correction des bugs, documentation, formation des équipes, déploiement en production avec Go/No-Go. Le système est prêt pour les utilisateurs réels.

ID	User Story	Pts	Équipe
US-043	Tests de charge k6 (500 req/s, p99 < 200ms)	8	DevOps
US-033	Politique de rétention documents	5	Dev 1
US-028	App mobile offline-first (MVP)	21	Dev 3+4+5
QUAL	Tests E2E complets (Playwright)	5	Dev 2
QUAL	Documentation API (OpenAPI enrichi)	3	Dev 1
QUAL	Formation utilisateurs (guide + vidéos)	2	PO
TOTAL		44	Capacité: 44 pts

PARTIE 5 — Estimation des Efforts & Récapitulatif

Récapitulatif complet : total des points par Epic, vélocité d'équipe, coût estimé en jours-homme, timeline et risques identifiés.

5.1 — Résumé par Epic

Epic	Nb US	Points Total	Must Have	Should Have	Could Have	Sprints concernés
EPIC-1 — Gestion Navires	6	32	29	3	0	S2
EPIC-2 — Dossiers Class.	7	47	24	13	10	S3
EPIC-3 — Certificats & Alertes	7	44	34	10	0	S4, S5
EPIC-4 — Inspections & NC	8	63	39	11	21	S4, S5
EPIC-5 — Documents	5	36	13	18	5	S2, S3, S5
EPIC-6 — Infrastructure DevOps	10	68	60	8	0	S1, tout
INFRASTRUCTURE SETUP	1	11	11	0	0	S1
QUALITÉ / FORMATION	3	10	10	0	0	S6
TOTAL	47	311	220	63	36	S1→S6

5.2 — Hypothèses & Vélocité d'Équipe

Paramètre	Valeur	Justification
Taille de l'équipe	7 personnes	5 développeurs full-stack, 1 ingénieur DevOps, 1 Product Owner
Durée d'un sprint	2 semaines	Standard Scrum, 10 jours ouvrés
Vélocité par sprint	~50 points	Équipe mid-senior, capacité 70% (cérémonies, congés, imprévus)
Durée totale	12 semaines	6 sprints × 2 semaines — hors phase de recette UAT
1 point =	~4h de travail	Tâche simple (1pt) à feature complexe (21pts)
Total points backlog	311 points	Dont 220 Must Have (71%), 63 Should (20%), 28 Could (9%)
Charge totale estimée	~155 jours-homme	311 pts × 4h / 8h/jour = 155,5 jours (tous profils confondus)
Coût indicatif	~310 000 €	Basé sur TJM moyen 1 000€/jour × 155 jours × 2 (overhead)

5.3 — Estimation Détaillée par Profil

Profil	Effectif	Sprints	Charge principale	Jours estimés	TJM	Coût
Dev Full-Stack Senior	2	S1→S6	Microservices core (vessel, classification, certificate)	50 j/pers = 100j	950 €/j	95 000 €
Dev Full-Stack Mid	2	S2→S6	Microservices (inspection, documents, notifications)	45 j/pers = 90j	850 €/j	76 500 €
Dev Full-Stack Junior	1	S3→S6	Frontend React, tests E2E, documentation	40j	700 €/j	28 000 €
Ingénieur DevOps	1	S1→S6	AKS, JFrog, ArgoCD, cert-manager, APIM, monitoring	60j	1100 €/j	66 000 €

Profil	Effectif	Sprints	Charge principale	Jours estimés	TJM	Coût
Product Owner	1	S1→S6	Backlog, acceptance criteria, formations, UAT	15j	800 €/j	12 000 €
TOTAL	7	—	—	~363j	—	277 500 €

5.4 — Coûts Infrastructure Azure (mensuel en production)

Ressource Azure	SKU / Config	Coût mensuel estimé
AKS — 6 nœuds Standard_D4s_v3	6 × 4 vCPU, 16 GB RAM	~1 200 €/mois
Azure Database PostgreSQL Flexible	Standard_D4s_v3, 128 GB SSD	~350 €/mois
JFrog Artifactory (Helm sur AKS)	Stockage Premium 200 GB	~150 €/mois (stockage)
Azure Blob Storage (documents)	LRS, ~500 GB, tier Hot	~20 €/mois
Azure Key Vault	Standard, ~10 000 opérations/j	~5 €/mois
Azure API Management	Standard — 1 unit	~700 €/mois
Azure DNS	Zone publique + requêtes	~5 €/mois
Azure Monitor / Log Analytics	Rétention 30j, ~10 GB/j logs	~80 €/mois
Azure Load Balancer (Ingress)	Standard	~20 €/mois
Azure DDoS Protection (optionnel)	Standard	~2 800 €/mois
TOTAL (sans DDoS)	—	~2 530 €/mois
TOTAL (avec DDoS)	—	~5 330 €/mois

5.5 — Registre des Risques

#	Risque	Probabilité	Impact	Mitigation
R01	Dépassement du budget infrastructure Azure si le volume de documents explose	Moyenne	Élevé	Mettre en place des budgets Azure + alertes à 80%. Utiliser Azure Blob tier Cool pour les documents > 1 an.
R02	Retard dans l'intégration AIS (API partenaire instable ou non documentée)	Élevée	Moyen	Mocker l'API AIS dès le Sprint 1. Prévoir un sprint de tampon. Contacter le fournisseur AIS en amont.
R03	Complexité du workflow d'approbation des dossiers sous-estimée (US-008 = 13pts)	Moyenne	Élevé	POC du moteur de workflow en Sprint 1 (hors scope MVP). Évaluer Camunda/Temporal si besoin.
R04	Certificats TLS non émis à temps (DNS propagation lente)	Faible	Moyen	Tester la configuration cert-manager en Sprint 1 avec letsencrypt-staging avant de passer en prod.
R05	Performance de la recherche full-text (US-032 = 13pts) insuffisante	Moyenne	Moyen	Dimensionner Azure Cognitive Search dès le début. Tester avec un jeu de 10 000 documents en Sprint 5.
R06	Rotation des secrets Key Vault casse les pods en production	Faible	Élevé	Configurer ESO refresh interval à 1h et tester le rolling restart. Alerter si ExternalSecret en erreur.
R07	US-028 (app mobile offline-first = 21pts) trop ambitieuse pour un seul sprint	Élevée	Moyen	Réduire au MVP : formulaire inspection offline + sync photo. Reporter les fonctionnalités avancées en V2.

■ Recommandations finales

1. Démarrer le Sprint 1 dès que l'infrastructure Azure est provisionnée (Étapes 1-4 du tutoriel). 2. Organiser un atelier de refinement des US-008 (workflow) et US-028 (mobile) avant le Sprint 3. 3. Planifier une démo tous les 2 sprints avec les armateurs pour valider les fonctionnalités. 4. Mettre en place un runbook d'exploitation dès le Sprint 4 (procédures de rollback ArgoCD, restauration BDD). 5. Prévoir une phase UAT de 2 semaines après le Sprint 6 avant la mise en production officielle.